

目录

• 文档说明

• 初始化配置🔥

• 加载组件

• 基础选项

• 设置回调函数🔥

• 自定义内容解析器

• 自定义初始聊天记录

• 扩展聊天工具栏🔥

• 创建聊天窗口🔥

• 普通聊天模式

• AI 对话模式

• 更多 API🔥

• 获取缓存信息

• 接受消息🔥

• 发送消息🔥

• 设置主题风格

• 获取当前聊天对象信息

• 添加联系人

• 移除联系人

• 打开申请联系人面板

• 打开添加好友面板

• 获取当前本地数据

• 设置消息盒子消息数

• 设置聊天面板最小化

• 设置当前聊天对象状态

• 设置好友在/离线状态

• 事件🔥

• 组件加载就绪的事件

• 聊天初始打开的事件

• 聊天窗口切换的事件

• 发送消息的事件🔥

• 切换在线状态的事件

• 签名被修改的事件

• 更换背景图的事件

• 查看群员的事件

• 延伸导读

• 数据交互方式

• WebSocket

• Fetch API 事件流

• 相关资料

https://dev.layuion.com/docs/layim/4/

LAYUION

LAYUI

扩展

主题



LayIM 4.x 主题文档

文档说明

当前文档适用于最新发布的 **LayIM 4.x** 电脑网页端版本，如果您正在使用的是其他版本，可点击以下链接进行切换：

4.x

3.x

3.x WAP 版

LayIM 是一套网页端聊天系统（WebIM）的纯静态主题，基于 Layui 组件库实现。采用原生态的 JavaScript, HTML, CSS 编写，只包含前端 UI（界面）和相关的模拟示例，不具备后台程序以数据库存储等完整功能。因此实际使用时，需要自行对接后端或云服务（如：融云、环信）、大模型等平台。

初始化配置🔥

layim.config(options)

- 参数 options：基础配置项，类型：Object。支持的可选项详见下文。

加载组件

LayIM 实际上是作为 Layui 的扩展组件而存在，因此只需要按照 [Layui 第三方扩展组件](#) 的方式加载 layim 组件即可：

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>LayIM 测试</title>
    <link href="../layui/css/layui.css" rel="stylesheet">
  </head>
  <body>
    <script src="../layui/layui.js"></script>
    <script>
      // 初始配置并加载 LayIM 组件
      layui.config({
        layimPath: '../src/', // 配置 layim.js 所在目录
        layimResPath: '../src/res/', // layim 资源文件所在目录
      }).extend({
        layim: layui.cache.layimPath + 'layim' // 配置 layim 组件所在的路径
      }).use('layim', function(layim) {
        // 基础配置
```

1/27

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

项

```
layim.config({
  // 初始化数据, 可采用 URL 接口或直接赋值两种方式
  init: {
    url: '', // 若采用 URL, 则 init 可同 jQuery.ajax() 参数选

    // ...
  },
  // 设置 iframe 页面地址
  pageurl: {
    // 消息盒子页面地址。若不开启, 剔除该项即可
    msgbox: layui.cache.layimResPath + 'html/msgbox.html',
    // 发现页面地址。若不开启, 剔除该项即可
    find: layui.cache.layimResPath + 'html/find.html',
    // 通过函数设置聊天记录页面地址。若不开启, 剔除该项即可
    chatlog: function(data) {
      var receiver = data.receiver; // 当前聊天对象
      // 设置基础路径, 此处以默认模板为例
      var url = layui.cache.layimResPath +
        'html/chatlog.html';
      return url + '?type=' + receiver.type + '&id=' +
        receiver.id;
    },
  },
  // theme: 'dark', // 默认强制深色主题风格
  // 更多选项见下文
  // ...
});

// 更多操作详见下文
// ...

});
</script>
</body>
</html>
```

基础选项

以下为 layim.config(options) 支持的可选项。

选项	描述	类型	默认值
contactsPanel	用于设置联系人面板。支持以下值类型： 若值为 boolean 类型, 则表示 是否开启 联系人面板。如: <code>contactsPanel: false</code> // 不开启联系人	boolean Object	true

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

选项	描述	类型	默认值
	面板，一般用于 AI 对话或客服场景 • 若值为 Object 类型，则表示开启联系人面板，且支持以下可选项： • showFriend：是否显示好友列表，默认 true • showGroup：是否显示群聊列表，默认 true • minStatus：是否初始为最小化状态，默认 false • done：联系人面板显示后的回调函数，可用于进行一些自定义操作。 <pre>contactsPanel: { // showFriend: true, // showGroup: true, // minStatus: false, done: function(elem) { // 设置面板相对右下角的偏移坐标 elem.css({ margin: '-32px 0 0 -32px' }); } }</pre>		
init	初始化数据，可采用请求 url 接口或直接赋值两种方式。 1. 若请求 url 接口，则 init 可同 jQuery.ajax() 参数选项 <pre>layim.config({ init: { url: '' // 此时 init 可配置成员同 jQuery.ajax() 参数选项 }, // ... });</pre> 该 url 接口所返回的 response 应遵循如下数据格式。 <pre>{ "code": 0, "msg": "", "data": { "user": {}, "friend": [], "group": [] } }</pre>		

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

选项	描述	类型	默认值
	其中 data 字段成员对应的信息如下： • user：当前用户信息 • friend：好友列表 • group：群组列表 以下是 data 字段各成员对应的字段示例： <pre>{ "code": 0, "msg": "", "data": { "user": { "username": "测试者", "id": "100000", "avatar": "", "sign": "测试签名", "status": "online" }, "friend": [{ "groupname": "测试分组一", "id": 0, "list": [{</pre>		
	2. 若直接赋值，则 init 可直接设置好友列表、群聊列表、历史聊天列表对应的数据。 <pre>layim.config({ // 数据格式同上述 url 接口返回的 data 字段中的 数据格式 init: { user: { // 当前用户信息 "username": "测试者", // 用户名称 "id": "100000", // 用户 ID "avatar": "", // 用户头像 "sign": "测试签名", // 用户签名 "status": "online" // 用户状态 - online 在线 hide 隐身 }, friend: [], // 好友列表 group: [] // 群聊列表 }, // 其他选项在此省略 // ... });</pre>		
pageurl	设置 iframe 页面地址，默认不开启该选项。支持设置以下可选项： • msgbox：消息盒子页面地址 • find：发现页面地址 • chatlog：查看更多聊天记录的页面地址 可直接赋值 iframe 页面 url 或通过函数动态拼接，如： <pre>// 基础配置 layim.config({ pageurl: { // 消息盒子页面地址。若不开启，剔除该项即可</pre>		

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

选项	描述	类型	默认值
	<pre>msgbox: layui.cache.layimResPath + 'html/msgbox.html', // 发现页面地址。若不开启，剔除该项即可 find: layui.cache.layimResPath + 'html/find.html', // 通过函数设置聊天记录页面地址。若不开启， 剔除该项即可 chatlog: function(data) { var receiver = data.receiver; // 当前聊天对象 var url = layui.cache.layimResPath + 'html/chatlog.html'; return url + '?type='+ receiver.type + '&id='+ receiver.id; } }, // 其他选项在此省略 // ... });</pre>		
theme	设置初始主题模式。支持设置以下值： - theme: 'dark': 深色主题 - theme: 'light': 浅色主题（默认）	string	light
chatTimeout	接受对方聊天消息时，等待响应的最大时长 (ms)	number	10000
defaultAvatar	默认头像的图片路径	string	-
backgroundImage	设置初始背景图片。读取组件 ./res/images/background/ 目录，如： backgroundImage: '1.jpg'	string	-
voice	设置新消息提示的音频文件。读取组件的 ./res/voice/ 目录。	string	default.mp3
LOCAL_TABLE	本地存储的表名	string	layim
LOCAL_MAX_CHATLOG	存储在本地的最多聊天记录数	string	20

设置回调函数🔥

layim.callback(name, handle)

- 参数 name：内置回调函数名，类型：string。支持的可选值详见下表。

name	描述
contentParser	用于自定义内容解析器，以重置 LayIM 内置的简化版 Markdown 解析器。
chatlog	用于自定义聊天记录初始渲染。 支持返回 Promise，以便通过异步获取数据并返回。

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

- 参数 handle：回调函数自定义行为，类型：function。

该方法允许重置 LayIM 的一些内置行为，以实现一些自定义操作。

自定义内容解析器

LayIM 提供了一套简化版的 Markdown 解析器，仅支持一些常用的规则。如果需要自定义更强大的解析器，可使用以下方式：

```
/**
 * 自定义内容解析器
 * 常用的 markdown 解析器有：marked, markdown-it
 * 下面以使用 markdown-it 为例，文档详见：
 * https://markdown-it.github.io/markdown-it/value
 */
layim.callback('contentParser', content => {
  const md = markdownit({
    typographer: true,
    linkify: true,
    breaks: true
  });
  return md.render(content); // 将 markdown 字符渲染成 HTML
});
```

自定义初始聊天记录

LayIM 默认读取浏览器本地存储中的聊天记录，实际应用中，通常需要获取服务端聊天记录数据，并渲染到 LayIM 聊天面板。chatlog 回调函数支持返回 Promise，这意味着您可以通过异步获取数据，并返回给函数，LayIM 将会接受到该数据。

```
// 异步获取服务端聊天记录并返回。若不设置，则默认读取本地记录
layim.callback('chatlog', obj => {
  // console.log(obj); // 当前聊天对象的相关信息
  return new Promise((resolve, reject) => {
    // 纯静态模拟获取聊天记录数据，实际使用时可将下述 setTimeout 换成真实接口
    setTimeout(function() {
      // 通过接口获得的数据
      let data = [{
        "username": "我",
        "id": "100000",
        "avatar": "",
        "timestamp": 1480897882000,
        "content": "我方模拟记录 111",
        "user": true
      }, {
        "username": "test123",
        "id": 108101,
        "avatar": "",
        "timestamp": 1480897892000,
```

目录

- 文档说明
- 初始化配置 🧨
 - 加载组件
 - 基础选项
- 设置回调函数 🧨
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🧨
- 创建聊天窗口 🧨
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🧨
 - 获取缓存信息
 - 接受消息 🧨
 - 发送消息 🧨
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🧨
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🧨
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
"content": "对方模拟记录 111"
}, {
  "username": "test123",
  "id": 108101,
  "avatar": "",
  "timestamp": 1480897908000,
  "content": "注意：这只是一个静态的聊天记录示例，实际使用时，可根据 chatlog 回调函数返回的参数请求对应会话的聊天记录。"
}];
// 将获得的数据返回给 Promise resolve
resolve(data);
}, 500);
});
});
```

上述示例改成 jQuery Ajax 的请求方式为：

```
// 异步获取服务端聊天记录并返回。若不设置，则默认读取本地记录
layim.callback('chatlog', obj => {
  return new Promise((resolve, reject) => {
    $.get(url, response => {
      resolve(response.data); // 返回数据。data 数据格式见上述示例
    });
  });
});
```

扩展聊天工具栏 🧨

```
layim.extendChatTools(tools)
```

- 参数 tools：自定义工具栏，类型：Object。

该方法允许在聊天窗口灵活扩展任意的快捷工具，如：发送快捷语、上传图片或文件、插入视频或语音消息等。

```
// 扩展任意聊天工具栏
layim.extendChatTools([
  {
    name: 'shortcut', // 自定义工具名称
    title: '发送快捷语', // 工具标题
    icon: 'layui-icon-list', // 工具图标
    onClick: function(obj) { // 自定义工具事件操作
      layui.dropdown.render({
        elem: obj.elem,
        data: [
          {title: '写一篇 50 字左右的正能量小说'},
          {title: 'Layui table 的详细用法'},
          {title: '1+1=?'}
        ]
      });
    }
  }
]);
```

目录

- 文档说明
- 初始化配置 🍷
 - 加载组件
 - 基础选项
- 设置回调函数 🍷
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🍷
- 创建聊天窗口 🍷
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🍷
 - 获取缓存信息
 - 接受消息 🍷
 - 发送消息 🍷
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🍷
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🍷
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
show: true,

click: function(data) {
    // obj.insert(data.title); // 将文本插入到编辑器
    layim.sendMessage(data.title); // 直接发送消息
}

}))
}
},
{
    name: 'image',
    title: '上传图片',
    icon: 'layui-icon-picture-fine',
    onClick: 'file', // 此处的 file 为内置工具
    ajax: {} // 设置上传图片接口, 同 jQuery.ajax() 参数选项
},
{
    name: 'file',
    title: '上传文件',
    icon: 'layui-icon-layim-uploadfile',
    onClick: 'file',
    ajax: {} // 设置上传图片接口, 同 jQuery.ajax() 参数选项
},
{
    name: 'video',
    title: '插入视频 URL',
    icon: 'layui-icon-video',
    onClick: 'remoteMedia' // 此处的 remoteMedia 为内置工具
},
{
    name: 'audio',
    title: '语音消息',
    icon: 'layui-icon-mike',
    onClick: function(obj) { // 自定义事件操作
        layer.msg('2 秒后模拟发送语音消息', {
            time: 2000
        }, function() {
            var text = '!audio(URL)'; // 音频消息特定格式 | 视频格式: !video(URL)
            layim.sendMessage(text); // 发送消息
        });
    }
},
{
    name: 'code',
    title: '插入代码',
    icon: 'layui-icon-fonts-code',
    onClick: function(obj) { // 自定义事件操作
        layer.prompt({
            title: '插入代码',
            formType: 2
        }, function(text, index) {
            layim.sendMessage(text);
        });
    }
}
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
    }, function(text, index){
        layer.close(index);
        text = ['`', text, '`'].join('\n'); // 拼接成默认内容解析器支持的 code 结构
        obj.insert(text); // 将文本插入到编辑器
    });
}
}
]);
```

创建聊天窗口 🔥

layim.chat(data)

- 参数 data：聊天窗口信息，类型：Object。支持的可选项如下：

选项	描述	类型	默认值
id	聊天对象唯一标识	string	-
username	聊天对象的用户名称	string	-
type	聊天类型，可选值：friend group ai。分别为：单聊、群聊、AI 对话。这是一个重要的选项，它决定聊天窗口的类型，不同的类型，聊天窗口的交互模式、功能细节等有所不同。详见下文。	string	-
avatar	聊天对象的头像图片地址	string	-
new	是否始终重新创建聊天窗口	boolean	false
enableLocalChatlog	聊天窗口是否启用本地聊天记录	boolean	true
layer	重置 layer 组件选项（聊天窗口基于 layer 创建）。详见： layer 文档	Object	
success	聊天窗口创建成功后的回调函数	function	-

该方法一般放置在单击事件中执行。

普通聊天模式

type 选项值为 friend 或 group 时，聊天均为普通模式。用户每次发送消息后，通常需在 WebSocket 的消息事件中获取消息内容，再渲染到聊天窗口。

```
// 单聊模式
layim.chat({
    type: 'friend',
    id: '100001',
    username: '张三',
    avatar: ''
});
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
// 群聊模式
layim.chat({
  type: 'group',
  id: '101001',
  username: '测试群',
  avatar: ''
});

// 接受消息的操作，一般采用 websocket 的消息事件回调函数处理。可参考下文「延伸阅读」的相关文档。
// ...
```

AI 对话模式

type 选项值为 ai 时，即为 AI 对话模式。用户每次发送消息后，对话中会自动创建一个临时的 Pending 容器，以用于承载 AI 回复内容的后续渲染。此时一般可采用事件流（如 EventSource）方式即时获取信息，并以打字效果显示 AI 回复内容。当然，你也可以采用普通异步（如 jQuery Ajax）请求，等待 AI 内容请求完毕后，再进行一次性渲染。

```
// 初始化配置。AI 模式一般不必显示联系人面板
layim.config({
  contactsPanel: false, // 不开启联系人面板
  init: {
    user: { // 配置当前用户信息
      "username": "测试者", // 我的昵称
      "id": "100000" // 我的 ID
    }
  }
});

// 创建 AI 对话窗口
layim.chat({
  type: 'ai',
  id: 108001,
  username: '通义千问 AI 助手',
  avatar: ''
});

// 接受 AI 消息的操作，一般推荐采用 Fetch API 事件流方式获取。可参考下文「延伸阅读」的相关文档。
// ...
```

更多 API 🔥

API	描述
layim.config(options) 🔥	初始化配置
layim.callback(name, handle) 🔥	设置内置回调函数

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

API	描述
layim.extendChatTools(tools) 🔥	扩展聊天工具栏
layim.chat(data) 🔥	创建聊天窗口
layim.cache()	获取所有缓存数据
layim.getMessage(data) 🔥	接受消息
layim.sendMessage(rawContent) 🔥	发送消息
layim.on(eventName, callback) 🔥	事件
layim.theme(style)	设置主题风格
layim.currentChat(index)	获取当前聊天对象相关信息
layim.addContacts(data)	添加联系人到主面板列表
layim.removeContacts(data)	从主面板列表移除联系人
layim.openRequestContactsPanel(data)	打开申请联系人面板
layim.openAddFriendPanel(data)	打开添加好友面板
layim.getLocalData(?uid)	获取当前本地数据
layim.setMsgboxCount(num)	设置消息盒子的新消息数量
layim.setChatMin()	设置聊天窗口最小化状态
layim.setChatStatus(str)	设置当前聊天对象的状态说明
layim.setFriendStatus(id, status)	设置好友在线/离线状态

获取缓存信息

```
layim.cache()
```

该方法可获取 layim 所有缓存信息，包括： options , user 等。

```
var layimCache = layim.cache();
console.log(layimCache);
```

接受消息 🔥

```
layim.getMessage(data)
```

- 参数 data : 消息信息，类型： Object 。支持的可选项如下：

选项	描述	类型	默认值
type	消息来源者的类型，可选值： friend group ai ，同 layim.chat() 的 type。	string	-
id	消息来源者的 ID。若是单聊，则为当前聊天对象的用户 id；若是群聊，则为群组 id。	string	-

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

选项	描述	类型	默认值
uid	发送消息的用户 ID，一般来自群聊消息时使用。单聊或 AI 消息无需设置，因为 id 即 uid。	string	-
username	消息来源者的用户名。	string	-
avatar	消息来源者的头像图片地址。	string	-
content	消息内容。	string	-
timestamp	消息发送时间（毫秒数），若未传递，默认读取本地时间。	number	-
mid	消息 ID。一般用于扩展撤回或删除消息等操作。	string	-
system	是否为系统消息。	boolean	false
saveLocalChatlog	是否将消息保存到本地聊天记录。优先级低于 layim.chat() 的 enableLocalChatlog 选项。	boolean	true
finished	消息是否为最终完成状态。一般在事件流中使用，设置 finished: false 表示消息仍处在事件流加载状态。	string	true

该方法用于为当前聊天面板接收一条消息，该消息一般可通过 WebSocket 或其他数据交互的方式获取，具体可参考下文「延伸阅读」的相关文档。

```
// 接受单聊消息
layim.getMessage({
  type: 'friend',
  id: '100001',
  username: '张三',
  avatar: '',
  content: '你好，我是张三。',
  timestamp: 1725726505815
});

// 接受群聊消息
layim.getMessage({
  type: 'group',
  id: '101001', // 群组 id
  uid: '100001', // 发送消息的用户 ID
  username: '张三',
  avatar: '',
  content: '大家好，我是张三。',
  timestamp: 1725726505815
});

// 接受系统消息
layim.getMessage({
  system: true,
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
id: '100001',
type: "friend",
content: '张三 拍了拍你'
});

// 接受 AI 消息
layim.getMessage({
  type: 'ai',
  id: 108001,
  username: '通义千问 AI 助手',
  avatar: '',
  content: '你好，我是通义千问 AI 助手。',
  timestamp: 1725726505815,
  finished: true
});
```

发送消息 🔥

```
layim.sendMessage(rawContent)
```

- 参数 rawContent : 发送消息的原始内容。类型：string。

该方法用于向当前聊天面板发送一条静态消息，而向后端发送的消息则需要通过 Websocket 或其他数据交互的方式发送，具体可参考下文「延伸阅读」的相关文档。

```
layim.sendMessage('你好，这是一条*测试消息*。'); // 一般配合扩展工具使用栏事件使用
```

设置主题风格

```
layim.theme(style)
```

- 参数 style : 主题风格，类型：string。可选值：light（浅色模式，默认）、dark（深色模式）

该方法用于动态设置主题的浅色或深色风格。

```
layim.theme('light'); // 设置浅色模式
layim.theme('dark'); // 设置深色模式
```

获取当前聊天对象信息

```
layim.currentChat(index)
```

- 参数 index : 聊天面板的索引，若不填则自动取当前聊天面板。类型：number。

该方法用于获取当前会话面板的相关信息，包括聊天对象信息、面板元素等。

```
var chatInfo = layim.currentChat();
console.log(chatInfo);
```

目录

- [文档说明](#)
- [初始化配置](#) 🔥
 - [加载组件](#)
 - [基础选项](#)
- [设置回调函数](#) 🔥
 - [自定义内容解析器](#)
 - [自定义初始聊天记录](#)
- [扩展聊天工具栏](#) 🔥
- [创建聊天窗口](#) 🔥
 - [普通聊天模式](#)
 - [AI 对话模式](#)
- [更多 API](#) 🔥
 - [获取缓存信息](#)
 - [接受消息](#) 🔥
 - [发送消息](#) 🔥
 - [设置主题风格](#)
 - [获取当前聊天对象信息](#)
 - [添加联系人](#)
 - [移除联系人](#)
 - [打开申请联系人面板](#)
 - [打开添加好友面板](#)
 - [获取当前本地数据](#)
 - [设置消息盒子消息数](#)
 - [设置聊天面板最小化](#)
 - [设置当前聊天对象状态](#)
 - [设置好友在/离线状态](#)
- [事件](#) 🔥
 - [组件加载就绪的事件](#)
 - [聊天初始打开的事件](#)
 - [聊天窗口切换的事件](#)
 - [发送消息的事件](#) 🔥
 - [切换在线状态的事件](#)
 - [签名被修改的事件](#)
 - [更换背景图的事件](#)
 - [查看群员的事件](#)
- [延伸导读](#)
- [数据交互方式](#)
 - [WebSocket](#)
 - [Fetch API 事件流](#)
- [相关资料](#)

添加联系人

layim.addContacts(data)

- 参数 data：联系人信息，类型：Object。支持的可选项如下：

选项	描述	类型	默认值
type	联系人类型，可选值：friend group	string	-
id	联系人 ID	string	-
username	联系人名称	string	-
avatar	联系人头像图片地址	string	-
gid	联系人分组 ID，若 type 为 friend 时	string	-
sign	联系人签名	string	-

该方法用于添加联系人到主面板列表。

```
// 添加好友到主面板
layim.addContacts({
  type: 'friend',
  id: '100001',
  username: '张三',
  avatar: '',
  gid: 2,
  sign: '签名信息'
});

// 添加群组到主面板
layim.addContacts({
  type: 'group',
  id: '101001',
  username: '群组名称',
  avatar: '',
});
```

移除联系人

layim.removeContacts(data)

- 参数 data：联系人信息，类型：Object。支持的可选项如下：

选项	描述	类型
type	联系人类型，可选值：friend group	string
id	联系人 ID	string

该方法用于移除主面板联系人。

目录

- 文档说明
- 初始化配置 🧨
 - 加载组件
 - 基础选项
- 设置回调函数 🧨
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🧨
- 创建聊天窗口 🧨
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🧨
 - 获取缓存信息
 - 接受消息 🧨
 - 发送消息 🧨
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🧨
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🧨
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
// 移除好友
layim.removeContacts({
  type: 'friend',
  id: '100001'
});

// 移除群组
layim.removeContacts({
  type: 'group',
  id: '101001'
});
```

打开申请联系人面板

```
layim.openRequestContactsPanel(data)
```

- 参数 data：联系人信息，类型：Object。支持的可选项如下：

选项	描述	类型
type	联系人类型，可选值：friend group	string
username	联系人名称	string
avatar	联系人头像图片地址	string
onConfirm	点击确定时的回调函数	function

该方法用于打开申请添加联系人的面板，如申请添加好友、申请加入群组。

```
// 申请添加好友
layim.openRequestContactsPanel({
  type: 'friend',
  username: '张三',
  avatar: '',
  onConfirm: function(obj){
    console.log(obj.gid); // 好友分组 id
    console.log(obj.note); // 验证信息
    console.log(obj.index); // layer 弹层索引，用于关闭面板

    // 此处可发送通知对方的接口
    // ...
  }
});

// 申请加入群组
layim.openRequestContactsPanel({
  type: 'group',
  username: '群组名称',
  avatar: '',
  onConfirm: function(obj){
    console.log(obj.note); // 验证信息
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
console.log(obj.index); // layer 弹层索引，用于关闭面板

// 此处可发送通知对方的接口
// ...
}
})
```

打开添加好友面板

```
layim.openAddFriendPanel(data)
```

- 参数 data：联系人信息，类型：Object。支持的可选项如下：

选项	描述	类型
type	联系人类型，可选值：friend group	string
username	联系人名称	string
avatar	联系人头像图片地址	string
onConfirm	点击确定按钮时的回调函数	function
onCancel	点击取消按钮时的回调函数	function

该方法用于打开同意添加好友的确认面板。

```
// 模拟一个好友数据，实际开发时需通过后端接口获取
var friendData = {
  type: 'friend',
  id: '100001',
  username: '张三',
  avatar: '', // 头像图片地址
  sign: '测试签名'
};

// 打开同意添加好友面板
layim.openAddFriendPanel({
  type: friendData.type,
  username: friendData.username,
  avatar: friendData.avatar,
  onConfirm: function(obj) {
    console.log(obj.gid); // 好友分组 id
    console.log(obj.index); // layer 弹层索引，用于关闭面板

    // 此处可发送通知对方已经同意添加好友的接口
    // ...

    // 同意后，将好友添加到主面板
    layim.addContacts({
      type: friendData.type,
      username: friendData.username,
      avatar: friendData.avatar,
```

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
id: friendData.id, // 好友 ID
sign: friendData.sign, // 好友签名
gid: obj.gid, // 所在的分组 id
});

layer.close(obj.index); // 关闭面板
}
});
```

获取当前本地数据

```
layim.getLocalData(?uid)
```

- 参数 uid: 用户 ID, 一般可不填, 会自动读取 layim.cache().user.id。类型: string。

该方法用于获取当前用户的本地数据, 如: 本地聊天记录、本地历史聊天列表、好友分组展开状态等。

```
var localData = layim.getLocalData();
console.log(localData);
```

设置消息盒子消息数

```
layim.setMsgboxCount(num)
```

- 参数 num: 消息数量, 类型: number。

该方法用于在联系人面板底部的消息图标处显示消息数量。

```
layim.setMsgboxCount(6);
```

设置聊天面板最小化

```
layim.setChatMin()
```

无需参数。该方法用于设置聊天面板最小化状态。

设置当前聊天对象状态

```
layim.setChatStatus(str)
```

- 参数 str: 状态文本, 类型: string。

该方法用于给当前聊天对象的面板头部设置状态文本, 如: 对方输入中、离线等。

```
layim.setChatStatus('对方输入中');
```

设置好友在/离线状态

```
layim.setFriendStatus(id, status)
```

- 参数 id: 好友 ID, 类型: string。
- 参数 status: 状态, 可选值: online | offline, 类型: string。

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

该方法用于设置联系人面板中的好友在线或离线状态。

```
layim.setFriendStatus('100001', 'offline'); // 设置好友离线
layim.setFriendStatus('100001', 'online'); // 设置好友在线
```

事件 🔥

```
layim.on(eventName, callback)
```

- 参数 eventName：事件名称，类型：string。支持的事件详见下表。
- 参数 callback：回调函数，类型：function。

事件名称	描述
ready	组件加载就绪的事件
chatInit	聊天初始打开的事件
chatChange	聊天窗口切换的事件
sendMessage 🔥	发送消息的事件
online	切换在线状态的事件
sign	签名被修改的事件
backgroundImage	更换背景图的事件
viewMmembers	查看群员的事件

组件加载就绪的事件

组件初始化完毕后触发，函数返回当前所有缓存信息。

```
layim.on('ready', function(cache) {
  console.log(cache); // 查看返回信息
});
```

聊天初始打开的事件

聊天面板初始打开时触发，一般可用于插入初始聊天信息。函数返回当前聊天对象的相关信息。

```
layim.on('chatInit', function(obj) {
  const data = obj.data;
  const elem = obj.elem;

  console.log(obj); // 查看返回信息

  // 插入初始信息
  layim.getMessage({
    system: true,
    id: data.id,
```


目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
type: 'ai',
content: '通义千问 AI 助手为您服务',
saveLocalChatlog: false // 不将该消息保存到本地的聊天记录
});
})
```

聊天窗口切换的事件

聊天面板切换时触发，函数返回当前聊天对象的相关信息。

```
layim.on('chatChange', function(obj) {
    const data = obj.data;
    const elem = obj.elem;

    console.log(obj); // 查看返回信息
});
```

发送消息的事件 🔥

在聊天面板发送消息时触发，函数返回一个对象，包含当前用户数据、聊天对象数据。**该事件是聊天双方数据交互的核心，几乎必用。**

```
layim.on('sendMessage', function(data) {
    const user = data.user; // 当前用户的数据（我）
    const receiver = data.receiver; // 当前聊天对象的数据（好友、群聊、AI 等）
    const type = receiver.type; // 聊天类型

    // 执行发送消息的相关逻辑与接口，详细交互可参考下文「延伸阅读」的相关文档
    // ...

    // 以下以发送消息时进行静态的自动回复为例
    // 模拟一段自动回复的文本
    const autoReplayTexts = [
        '模拟消息 1',
        '模拟消息 2',
        '模拟消息 3',
        '模拟消息 4',
        '模拟消息 5',
        '模拟消息 6',
        '模拟消息 7',
        '模拟消息 8',
        '模拟消息 9'
    ];

    // 模拟接受好友的消息
    if (type === 'friend') {
        layim.getMessage({
            ...receiver,
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
        content: autoReplayTexts[Math.floor(Math.random() *
autoReplay.length)] // 获取随机文本
    });
    }
  });
}
```

切换在线状态的事件

点击联系人面板的在线状态列表时触发，函数返回当前用户的在线状态值。

```
layim.on('online', function(obj) {
  console.log(obj.status); // 状态值
})
```

签名被修改的事件

点击联系人面板的签名区域，在输入框失去焦点后，若文本出现修改时触发，函数返回修改后的签名内容。

```
layim.on('sign', function(obj) {
  console.log(obj.value); // 修改后的签名内容
});
```

更换背景图的事件

点击联系人面板底部的设置按钮打开设置面板，在设置面板中点击「背景底图」时触发。函数返回当前背景图的相关信息。

```
layim.on('backgroundImage', function(obj) {
  console.log(obj.filename); // 图片文件名
  console.log(obj.src); // 图片路径
});
```

查看群员的事件

点击群聊面板头部的查看群员的图标时触发，函数返回当前群聊对象的信息。利用该事件可自主实现群员列表的数据渲染。

```
layim.on('viewMmembers', function(data) {
  var receiver = data.receiver; // 当前聊天对象的数据（此处一般为当前群组）

  console.log(data); // 查看函数的返回信息

  // 请求群员接口
  data.ajax({
    url: '', // 当前群组的群员列表接口
    data: {
      id: receiver.id
    }
  });
});
```

目录

- [文档说明](#)
- [初始化配置](#) 🔥
 - [加载组件](#)
 - [基础选项](#)
- [设置回调函数](#) 🔥
 - [自定义内容解析器](#)
 - [自定义初始聊天记录](#)
- [扩展聊天工具栏](#) 🔥
- [创建聊天窗口](#) 🔥
 - [普通聊天模式](#)
 - [AI 对话模式](#)
- [更多 API](#) 🔥
 - [获取缓存信息](#)
 - [接受消息](#) 🔥
 - [发送消息](#) 🔥
 - [设置主题风格](#)
 - [获取当前聊天对象信息](#)
 - [添加联系人](#)
 - [移除联系人](#)
 - [打开申请联系人面板](#)
 - [打开添加好友面板](#)
 - [获取当前本地数据](#)
 - [设置消息盒子消息数](#)
 - [设置聊天面板最小化](#)
 - [设置当前聊天对象状态](#)
 - [设置好友在/离线状态](#)
- [事件](#) 🔥
 - [组件加载就绪的事件](#)
 - [聊天初始打开的事件](#)
 - [聊天窗口切换的事件](#)
 - [发送消息的事件](#) 🔥
 - [切换在线状态的事件](#)
 - [签名被修改的事件](#)
 - [更换背景图的事件](#)
 - [查看群员的事件](#)
- [延伸导读](#)
- [数据交互方式](#)
 - [WebSocket](#)
 - [Fetch API 事件流](#)
- [相关资料](#)

```
},
    success: function(res) {
        data.render(res.data); // 渲染群员列表
    }
});
});
```

延伸导读

以下是延伸的文档，用于辅助 LayIM 的使用，并非组件本身的内容。

数据交互方式

LayIM 作为一套用于 Web 聊天的纯静态的 UI 主题，本身不具备与服务端的数据交互能力。因此，在实际开发时，一般需自主实现数据交互等完整的应用功能。

· 以下示例仅供参考 ·

WebSocket

以下示例一般用于 **普通聊天模式**，实际开发时需自行进行修改。如果您选择自主对接 WebSocket 服务，亦可详细参考 [WebSocket 官方文档](#)。

```
/**
 * 本示例假设您已充分阅读上述全部文档
 */

// 基础配置
layim.config({
    // 初始化数据，可采用 URL 接口或直接赋值两种方式
    init: {
        url: '', // 若采用 URL，则 init 可同 jQuery.ajax() 参数选项
        // ...
    },
    // 其他选项在此省略
    // ...
});

// 创建一个 WebSocket 实例
const socket = new WebSocket('ws://localhost:8090');

// ws 连接成功事件
socket.onopen = function() {
    console.log('WebSocket connection successful.');
```

目录

- [文档说明](#)
- [初始化配置](#) 🔥
 - [加载组件](#)
 - [基础选项](#)
- [设置回调函数](#) 🔥
 - [自定义内容解析器](#)
 - [自定义初始聊天记录](#)
- [扩展聊天工具栏](#) 🔥
- [创建聊天窗口](#) 🔥
 - [普通聊天模式](#)
 - [AI 对话模式](#)
- [更多 API](#) 🔥
 - [获取缓存信息](#)
 - [接受消息](#) 🔥
 - [发送消息](#) 🔥
 - [设置主题风格](#)
 - [获取当前聊天对象信息](#)
 - [添加联系人](#)
 - [移除联系人](#)
 - [打开申请联系人面板](#)
 - [打开添加好友面板](#)
 - [获取当前本地数据](#)
 - [设置消息盒子消息数](#)
 - [设置聊天面板最小化](#)
 - [设置当前聊天对象状态](#)
 - [设置好友在/离线状态](#)
- [事件](#) 🔥
 - [组件加载就绪的事件](#)
 - [聊天初始打开的事件](#)
 - [聊天窗口切换的事件](#)
 - [发送消息的事件](#) 🔥
 - [切换在线状态的事件](#)
 - [签名被修改的事件](#)
 - [更换背景图的事件](#)
 - [查看群员的事件](#)
- [延伸导读](#)
- [数据交互方式](#)
 - [WebSocket](#)
 - [Fetch API 事件流](#)
- [相关资料](#)

```
// LayIM 发送消息事件
layim.on('sendMessage', function(data) {
    // 当前用户的数据（我），包含了消息内容。用于告知服务器消息来自谁
    const user = data.user;
    // 当前聊天对象的数据（好友、群聊等），用于告知服务器消息应该推送给谁
    const receiver = data.receiver;

    // 向 ws 服务器发送消息
    socket.send(JSON.stringify(data));
});

// ws 服务器接收消息事件
socket.onmessage = function(event) {
    /**
     * 解析数据
     * 若 data 是 socker.send() 时传递的原样数据，作为消息接收者，此处需互
     换 user 和 receiver 的关系
     */
    const data = JSON.parse(event.data);
    const user = data.user; // 此处为消息来源者的数据（即好友）
    const receiver = data.receiver; // 此处为消息接收者的数据（即我）

    // 将接受到的数据传递给 layim，并渲染到聊天目绵版
    layim.getMessage({
        ...data.user, // 消息来源者的数据集，包含了消息内容
        type: receiver.type,
        uid: receiver.id,
        user: null
    });
};

// ws 其他事件，如 onclose, onerror 详见 WebSocket 官方文档。此处不做
列举。
```

Fetch API 事件流

以下示例一般用于 **AI 对话模式**，实际开发时需自行进行修改。

```
/**
 * 本示例假设您已充分阅读上述全部文档
 */

// 基础配置
layim.config({
    contactsPanel: false, // AI 场景一般不开启联系人面板
    init: {
        user: { // 配置当前用户信息
            "username": "测试者", // 我的昵称
            "id": "100000" // 我的 ID
        }
    }
});
```

目录

- 文档说明
- 初始化配置 🔥
 - 加载组件
 - 基础选项
- 设置回调函数 🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🔥
- 创建聊天窗口 🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🔥
 - 获取缓存信息
 - 接受消息 🔥
 - 发送消息 🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
    }
  }
});

/**
 * 自定义内容解析器
 * AI 一般返回 markdown 格式的内容，您可借助第三方 markdown 解析器渲染
AI 返回的内容
 *
 * 注意：
 * 1. markdown 解析器应禁用 HTML 标签（通常为默认），避免 XSS，确保返回
安全的 HTML 内容
 * 2. 若不采用 markdown 解析器，请务必自行转义 XSS 字符
 *
 * 下面以使用 markdown-it 为例
 * @see https://markdown-it.github.io/markdown-it/value
 */
layim.callback('contentParser', content => {
  const md = markdownit({
    typographer: true,
    linkify: true,
    breaks: true
  });
  return md.render(content); // 将 markdown 字符渲染成 HTML
});

// LayIM 发送消息事件
layim.on('sendMessage', function(data) {
  const user = data.user; // 当前用户的数据（即我），包含了消息内容。
  const receiver = data.receiver; // 当前聊天对象的数据（即 AI）

  /**
   * 以下分别演示采用「事件流」和「jQuery Ajax」方式请求阿里云「通义千问」
接口
   * 阿里「通义千问」@see
https://help.aliyun.com/zh/dashscope/developer-reference/
   * 百度「千帆大模型」@see
https://cloud.baidu.com/doc/WENXINWORKSHOP/s/6ltgkzya5
   *
   * 请注意 ⚡：
   * 1. 以下代码仅用于演示，请勿直接用于生产环境。
   * 2. 实际开发时，请务必将请求大模型的接口代码写在后端，避免在浏览器端
直接暴露 API_KEY，SECRET_KEY。
   */
  if (receiver.id === 'Qwen-ai') {
    async function requestQwen (callback) {
      // 模型接口
      const API_URL =
'https://dashscope.aliyuncs.com/api/v1/services/aigc/text-
generation/generation';
```

目录

- 文档说明
- 初始化配置🔥
 - 加载组件
 - 基础选项
- 设置回调函数🔥
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏🔥
- 创建聊天窗口🔥
 - 普通聊天模式
 - AI 对话模式
- 更多 API🔥
 - 获取缓存信息
 - 接受消息🔥
 - 发送消息🔥
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件🔥
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件🔥
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸导读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

取

```
const API_KEY = ''; // 模型密钥，可通过阿里云 DashScope 平台获

const API_MODEL = 'qwen2-1.5b-instruct'; // 模型名称。此处以
qwen2 免费的轻量级模型为例

// 交互方式
const QWEN_REQUEST_MODE = 'stream'; // stream 或 ajax

// 基础参数
const options = {
  method: 'POST',
  headers: {
    "Authorization": `Bearer ${API_KEY}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    model: API_MODEL, // 模型
    input: {
      messages: messages[rid]
    },
    parameters: {
      temperature: 1.3,
      top_p: 0.9,
      max_tokens: 1500,
    }
  })
};

// 事件流交互方式
if (QWEN_REQUEST_MODE === 'stream') { // 事件流方式
  // 此处 fetchEventSource 方法采用的是 fetch-event-source 插
  件，需自行引入
  // 用法详见: https://www.npmjs.com/package/@sentool/fetch-
  event-source

  const eventSource = await fetchEventSource(API_URL, {
    ...options,
    onopen(response) {
      if (!response.ok) {
        throw new Error(`URL ${response.status}
        ${response.statusText}`)
      }
    },
    onmessage(event) {
      // console.log(event);
      const { data, id } = event;
      const { output } = data;
      const finished = output.finish_reason === 'stop';

      let text = output.text + ((finished || id % 2) ? '
      : ' _');
```

目录

- 文档说明
- 初始化配置 🍷
 - 加载组件
 - 基础选项
- 设置回调函数 🍷
 - 自定义内容解析器
 - 自定义初始聊天记录
- 扩展聊天工具栏 🍷
- 创建聊天窗口 🍷
 - 普通聊天模式
 - AI 对话模式
- 更多 API 🍷
 - 获取缓存信息
 - 接受消息 🍷
 - 发送消息 🍷
 - 设置主题风格
 - 获取当前聊天对象信息
 - 添加联系人
 - 移除联系人
 - 打开申请联系人面板
 - 打开添加好友面板
 - 获取当前本地数据
 - 设置消息盒子消息数
 - 设置聊天面板最小化
 - 设置当前聊天对象状态
 - 设置好友在/离线状态
- 事件 🍷
 - 组件加载就绪的事件
 - 聊天初始打开的事件
 - 聊天窗口切换的事件
 - 发送消息的事件 🍷
 - 切换在线状态的事件
 - 签名被修改的事件
 - 更换背景图的事件
 - 查看群员的事件
- 延伸阅读
- 数据交互方式
 - WebSocket
 - Fetch API 事件流
- 相关资料

```
callback?.(text, finished);

// 将回复内容加入到多轮会话
if (finished) {
    messages[rid]?.push({
        role: 'assistant',
        content: text
    });
},
onerror(error) {
    callback(`**Error**: ${error}`);
}
});
// console.log(eventSource);
} else if (QWEN_REQUEST_MODE === 'ajax') { // 传统 jQuery
    Ajax 交互方式
    // 一般地，如果没有什么特别要求，Ajax 交互方式相对会更加简单
    options.data = options.body;
    delete options.body;

    layui.$.ajax({
        ...options,
        url: API_URL,
        success(data) {
            // console.log(data);
            const { output } = data;
            const finished = output.finish_reason === 'stop';
            let text = output.text;

            callback?.(text, finished);

            // 将回复内容加入到多轮会话
            messages[rid]?.push({
                role: 'assistant',
                content: text
            });
        },
        error(error) {
            callback(`**Error**: ${error.responseText}`);
        }
    });
}

// 多轮会话
const rid = receiver.id;
const messages = layim.chat.messages = layim.chat.messages
|| {
```


目录

- [文档说明](#)
- [初始化配置](#) 🔥
 - [加载组件](#)
 - [基础选项](#)
- [设置回调函数](#) 🔥
 - [自定义内容解析器](#)
 - [自定义初始聊天记录](#)
- [扩展聊天工具栏](#) 🔥
- [创建聊天窗口](#) 🔥
 - [普通聊天模式](#)
 - [AI 对话模式](#)
- [更多 API](#) 🔥
 - [获取缓存信息](#)
 - [接受消息](#) 🔥
 - [发送消息](#) 🔥
 - [设置主题风格](#)
 - [获取当前聊天对象信息](#)
 - [添加联系人](#)
 - [移除联系人](#)
 - [打开申请联系人面板](#)
 - [打开添加好友面板](#)
 - [获取当前本地数据](#)
 - [设置消息盒子消息数](#)
 - [设置聊天面板最小化](#)
 - [设置当前聊天对象状态](#)
 - [设置好友在/离线状态](#)
- [事件](#) 🔥
 - [组件加载就绪的事件](#)
 - [聊天初始打开的事件](#)
 - [聊天窗口切换的事件](#)
 - [发送消息的事件](#) 🔥
 - [切换在线状态的事件](#)
 - [签名被修改的事件](#)
 - [更换背景图的事件](#)
 - [查看群员的事件](#)
- [延伸阅读](#)
- [数据交互方式](#)
 - [WebSocket](#)
 - [Fetch API 事件流](#)
- [相关资料](#)

```
[rid]: []
};
messages[rid] = messages[rid] || []
messages[rid]?.push({
  role: 'user',
  content: user.content
});

// 获得 AI 回复内容
requestQwen((content, finished) => {
  // layim 接受对话消息
  layim.getMessage({
    ...receiver,
    content: content,
    finished
  });
});
}
});

// 聊天面板初始打开的事件
layim.on('chatInit', function(obj) {
  const data = obj.data;
  const elem = obj.elem;
  if (data.type === 'ai') {
    // 每次刷新页面，即提示为新的会话
    const lastItem = elem.find('.layim-chat-main>ul>li').last();
    const existMessages = layim.chat.messages &&
layim.chat.messages[data.id];
    if (!existMessages && !lastItem.hasClass('layim-chat-
system')) {
      layim.getMessage({
        system: true,
        id: data.id,
        type: 'ai',
        content: '以下为新的会话',
      });
    }
  }
});

/**
 * 其他业务操作
 */
const { $, util, form } = layui;

// 在按钮的点击事件中创建一个 AI 对话面板
$('#btn-id').on('click', function() {
  layim.chat({
```

目录

- [文档说明](#)
- [初始化配置](#) 🔥
 - [加载组件](#)
 - [基础选项](#)
- [设置回调函数](#) 🔥
 - [自定义内容解析器](#)
 - [自定义初始聊天记录](#)
- [扩展聊天工具栏](#) 🔥
- [创建聊天窗口](#) 🔥
 - [普通聊天模式](#)
 - [AI 对话模式](#)
- [更多 API](#) 🔥
 - [获取缓存信息](#)
 - [接受消息](#) 🔥
 - [发送消息](#) 🔥
 - [设置主题风格](#)
 - [获取当前聊天对象信息](#)
 - [添加联系人](#)
 - [移除联系人](#)
 - [打开申请联系人面板](#)
 - [打开添加好友面板](#)
 - [获取当前本地数据](#)
 - [设置消息盒子消息数](#)
 - [设置聊天面板最小化](#)
 - [设置当前聊天对象状态](#)
 - [设置好友在/离线状态](#)
- [事件](#) 🔥
 - [组件加载就绪的事件](#)
 - [聊天初始打开的事件](#)
 - [聊天窗口切换的事件](#)
 - [发送消息的事件](#) 🔥
 - [切换在线状态的事件](#)
 - [签名被修改的事件](#)
 - [更换背景图的事件](#)
 - [查看群员的事件](#)
- [延伸导读](#)
- [数据交互方式](#)
 - [WebSocket](#)
 - [Fetch API 事件流](#)
- [相关资料](#)

```
type: 'ai',
id: 'Qwen-ai', // 唯一的 ID
username: '通义千问 AI 助手',
avatar: '', // AI 头像
});
});
```

AI 交互的详细实例也可以直接参考 LayIM 下载包中提供的 demo/ai.html 文件。

译

相关资料

- [WebSocket API](#)
- [Fetch API](#)
- [fetch-event-source](#)
- [阿里云 DashScope](#)
- [百度千帆大模型](#)

若文档所提及的功能仍然无法满足，您可以直接对主题源代码进行修改。最后，祝君能熟练运用 LayIM。